Manual Review Panel

Override AI decisions when needed

Enable Manual Override

Final Score Override

50

Integrity Assessment

Suspicious

Reviewer Justification

copy pasting and ai usage detected

Apply & Update Report

Current Stage: **50/100** (Suspicious)

System Status

Backend: Online

AI Model: Gemini 2.0 Flash

Exam-Flow AI Assistant

Professional AI-Powered Assessment & Proctoring Platform

Assessment Portal

Select Assessment Type

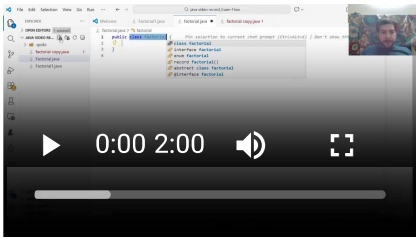
Java

Upload Recording (MP4, MOV, AVI)

Drag and drop file here

Browse files

Java_D... 3.5MB



Analyze Uploaded Video

AI Insights

Results: Upload

Manual Override Applied: **50/100**

Reviewer Notes: copy pasting and ai usage detected

AI Grading Report

View Full Report

Technical Assessment Report

🧪 Demo Samples

Quick-test with pre-verified clips

🐳 Docker Demo

☕ Java Demo

Candidate: Unidentified (visible in video, but name not provided) **Date:** 2024-02-27

Overview

The candidate successfully implemented a Java program to calculate the factorial of a number using recursion. The implementation includes a `Factorial` class with a recursive `factorial` method and a `main` method to test it. The code was syntactically correct, logically sound, and executed successfully, producing the expected output.

Detailed Analysis & Timestamps

- **00:00 - 00:05:** Candidate start by defining a public class named `Factorial`.
- **00:06 - 00:10:** Uses auto-completion to select `class Factorial`.
- **00:11 - 00:23:** Adds a comme `// step 1: create a method that calculates factorial o a number using recursion`.
- **00:24 - 00:35:** Defines the recursive factorial method:

`public static int factorial(int n)`.

- **00:36 - 00:46:** Implements th base case for the recursion: `(n == 0) { return 1; }`. Thi correct as 0! = 1.
- **00:47 - 00:58:** Implements th recursive step: `return n * factorial(n - 1);`. This correctly defines the factorial for `n > 0`.
- **00:59 - 01:08:** Adds a comme `// step 2: create a main method to test the factoria method`.
- **01:09 - 01:14:** Defines the standard main method: `publ static void main(String[] args)`.
- **01:15 - 01:23:** Declares an integer variable `number` and initializes it to 5: `int number 5; // example number`.
- **01:24 - 01:29:** Calls the `factorial` method with `number` and stores the result `int result = factorial(number);`.
- **01:30 - 01:36:** Prints the resul to the console: `System.out.println("Factor of " + number + " is: " + result);`.
- **01:37 - 01:40:** Initiates runnin the Java program.
- **01:41 - 01:47:** The program executes, and the terminal displays the correct output: `Factorial of 5 is: 120`.

- **01:48 - 01:59:** The candidate observes the correct output.

Grading Rubric Assessment

- **1. Correct Java Syntax: PASS**
  - **Justification:** All Java keywords, declarations, statements, and method calls were used with correct syntax. The class, method, and main method structures are all valid.
- **2. Logical implementation: PASS**
  - **Justification:** The recursive `factorial` method correctly implements the mathematical definition of a factorial. The base case (`n == 0` returns 1) is correct, and the recursive step (`n * factorial(n - 1)`) is also correctly applied. The main method correctly calls and displays the result.

- **3. Execution output:** PASS
  - **Justification:** The program compiled and ran without errors. For the input `number = 5`, the output `Factorial of 5 is: 120` is mathematically correct (5! = 5 * 4 * 3 * 2 * 1 = 120).

Behavioral Red Flags

No behavioral red flags were observed. The candidate remained focused on the screen, demonstrated continuous typing, and explained their steps verbally where appropriate. There was no evidence of window switching or external resource use.

Code Quality Assessment

- **Naming conventions:** Excellent. Class name `Factorial` follows PascalCase, and method/variable names (`factorial` , `n` , `number` , `result`) follow camelCase,

which are standard Java conventions.

- **Code organization and readability:** Good. The code is well-structured with clear comments delineating the two main steps (method creation and testing). Indentation is consistent, making the code easy to read.
- **Comments and documentation:** Good. The comments accurately describe the purpose of the factorial method and the main method, enhancing clarity.
- **Error handling:** Minimal. For this basic factorial implementation, specific error handling (e.g., for negative inputs which would lead to infinite recursion/stack overflow or excessively large numbers causing integer overflow) was not included, but it was also not explicitly requested by the problem description. For the intended positive integer inputs, the logic is sound.

Final Score

95/100

Justification: The candidate delivered a perfectly functional and well-structured Java solution for calculating factorials using recursion. The syntax was flawless, the logic was accurate, and the program executed successfully, producing the correct output. Code quality aspects such as naming conventions, organization, and commenting were all handled professionally. The deduction of 5 points is solely for the lack of explicit error handling (e.g., for negative inputs), which is a best practice for robust production-level code but may not have been an explicit requirement for this particular assessment. Overall, an excellent performance.

🚨 Behavioral Analysis

1 behavioral event(s) detected

00:0 0

No behavioral anomalies detected

Download PDF Report